第二次作业：CNN实现SVHN图像分类任务

In [1]:
```
import os
import random
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
```

In [2]:
```
# ========================
# 0. 全局设置
# ========================
SEED = 42
BATCH_SIZE = 128
LEARNING_RATE = 1e-3
NUM_EPOCHS = 15   # 可先改成 5 试跑，正式训练再改回 15 或更高
NUM_CLASSES = 10
MODEL_SAVE_PATH = "svhn_cnn_model.pth"

TRAIN_MAT_PATH = "train_32x32.mat"
TEST_MAT_PATH = "test_32x32.mat"

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "Arial Unicode M
plt.rcParams["axes.unicode_minus"] = False
```

```
Using device: cpu
```

In [3]:
```
# ========================
# 1. 读取数据
# ========================
if not os.path.exists(TRAIN_MAT_PATH):
    raise FileNotFoundError(f"未找到文件: {TRAIN_MAT_PATH}")

if not os.path.exists(TEST_MAT_PATH):
    raise FileNotFoundError(f"未找到文件: {TEST_MAT_PATH}")

train_mat = loadmat(TRAIN_MAT_PATH)
test_mat = loadmat(TEST_MAT_PATH)

print("train_mat keys:", train_mat.keys())
print("test_mat keys:", test_mat.keys())
```

```
X_train_raw = train_mat["X"]   # shape: (32, 32, 3, N)
y_train_raw = train_mat["y"]   # shape: (N, 1)

X_test_raw = test_mat["X"]
y_test_raw = test_mat["y"]

print("X_train_raw shape:", X_train_raw.shape)
print("y_train_raw shape:", y_train_raw.shape)
print("X_test_raw shape:", X_test_raw.shape)
print("y_test_raw shape:", y_test_raw.shape)
```

```
train_mat keys: dict_keys(['__header__', '__version__', '__globals__', 'X', 'y'])
test_mat keys: dict_keys(['__header__', '__version__', '__globals__', 'X', 'y'])
X_train_raw shape: (32, 32, 3, 73257)
y_train_raw shape: (73257, 1)
X_test_raw shape: (32, 32, 3, 26032)
y_test_raw shape: (26032, 1)
```

In [4]:

```
# ========================
# 2. 数据预处理
# ========================
y_train = y_train_raw.squeeze()
y_test = y_test_raw.squeeze()

# SVHN 中标签 10 表示数字 0
y_train[y_train == 10] = 0
y_test[y_test == 10] = 0

# 图像维度从 (32, 32, 3, N) 转成 (N, 32, 32, 3)
X_train = np.transpose(X_train_raw, (3, 0, 1, 2))
X_test = np.transpose(X_test_raw, (3, 0, 1, 2))

print("处理后 X_train shape:", X_train.shape)
print("处理后 y_train shape:", y_train.shape)
print("处理后 X_test shape:", X_test.shape)
print("处理后 y_test shape:", y_test.shape)

print("训练集类别分布：", np.bincount(y_train))
print("测试集类别分布：", np.bincount(y_test))
```

```
处理后 X_train shape: (73257, 32, 32, 3)
处理后 y_train shape: (73257,)
处理后 X_test shape: (26032, 32, 32, 3)
处理后 y_test shape: (26032,)
训练集类别分布： [ 4948 13861 10585  8497  7458  6882  5727  5595  5045  4659]
测试集类别分布： [1744 5099 4149 2882 2523 2384 1977 2019 1660 1595]
```

In [5]:

```
# ========================
# 3. 可视化部分训练样本
# ========================
plt.figure(figsize=(10, 6))
for i in range(20):
    plt.subplot(4, 5, i + 1)
    plt.imshow(X_train[i])
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")
plt.suptitle("SVHN 训练集样本示例")
plt.tight_layout()
plt.show()
```

In [6]:
```
# ========================
# 4. 构建 Dataset
# ========================
class SVHNDataset(Dataset):
    def __init__(self, images, labels, transform=None):
        self.images = images
        self.labels = labels.astype(np.int64)
        self.transform = transform

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        image = self.images[idx]
        label = self.labels[idx]

        if self.transform is not None:
            image = self.transform(image)

        return image, label
```

In [7]:
```
# ========================
# 5. 数据增强 / 标准化
# ========================
train_transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.RandomCrop(32, padding=2),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5],
                         std=[0.5, 0.5, 0.5])
])

test_transform = transforms.Compose([
    transforms.ToPILImage(),
```

```
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5],
                         std=[0.5, 0.5, 0.5])
])

train_dataset = SVHNDataset(X_train, y_train, transform=train_transform)
test_dataset = SVHNDataset(X_test, y_test, transform=test_transform)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, nu
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num

print("训练批次数:", len(train_loader))
print("测试批次数:", len(test_loader))
```

```
训练批次数: 573
测试批次数: 204
```

In [8]:

```
# ========================
# 6. 构建 CNN 模型
# ========================
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super(SimpleCNN, self).__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

model = SimpleCNN(num_classes=NUM_CLASSES).to(device)
print(model)
```

```
SimpleCNN(
  (features): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stat
s=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=Fals
e)
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stat
s=True)
    (6): ReLU()
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=Fals
e)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
    (10): ReLU()
    (11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=Fal
se)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=2048, out_features=256, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=256, out_features=10, bias=True)
  )
)
```

In [9]:

```python
# ========================
# 7. 损失函数与优化器
# ========================
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

In [10]:

```python
# ========================
# 8. 训练 / 测试函数
# ========================
def train_one_epoch(model, dataloader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in dataloader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

```
    epoch_loss = running_loss / total
    epoch_acc = correct / total
    return epoch_loss, epoch_acc


def evaluate(model, dataloader, criterion, device):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            running_loss += loss.item() * images.size(0)

            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    epoch_loss = running_loss / total
    epoch_acc = correct / total
    return epoch_loss, epoch_acc
```

In [11]:

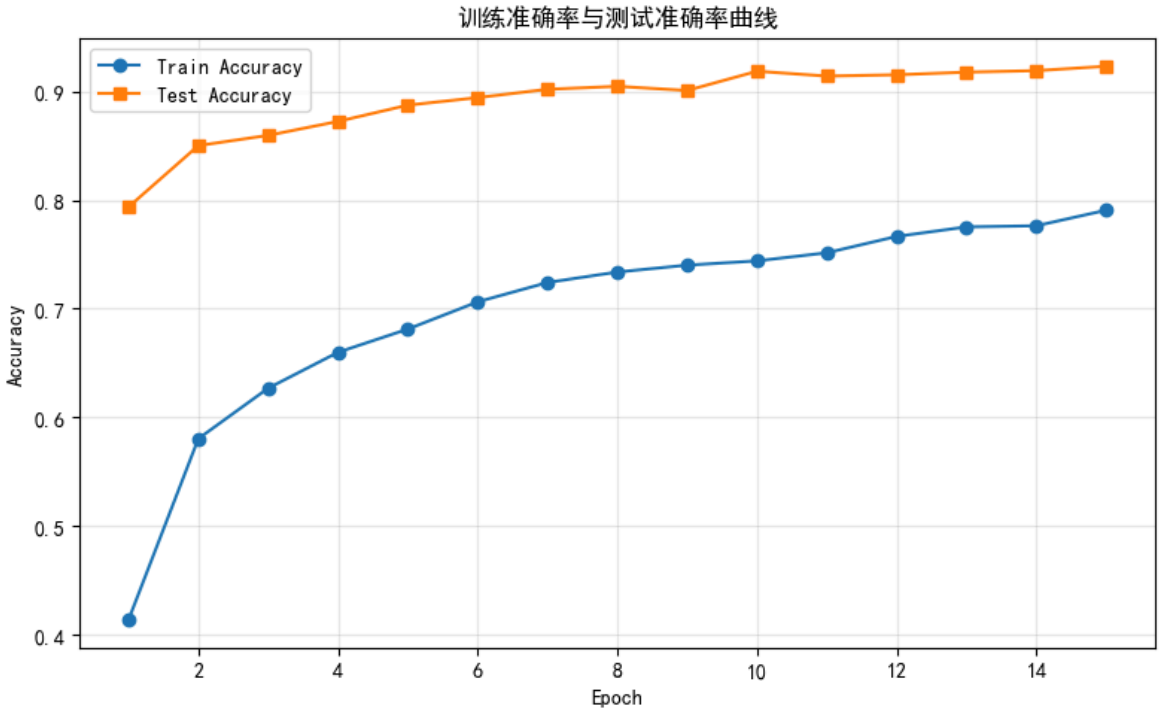
```
# ========================
# 9. 模型训练
# ========================
train_losses = []
test_losses = []
train_accuracies = []
test_accuracies = []

best_test_acc = 0.0

for epoch in range(NUM_EPOCHS):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, opti
    test_loss, test_acc = evaluate(model, test_loader, criterion, device)

    train_losses.append(train_loss)
    test_losses.append(test_loss)
    train_accuracies.append(train_acc)
    test_accuracies.append(test_acc)

    print(f"Epoch [{epoch+1}/{NUM_EPOCHS}]")
    print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f}")
    print(f"Test  Loss: {test_loss:.4f} | Test  Acc: {test_acc:.4f}")
    print("-" * 50)

    if test_acc > best_test_acc:
        best_test_acc = test_acc
        torch.save(model.state_dict(), MODEL_SAVE_PATH)

print("训练完成。")
```

```
print(f"最佳测试准确率: {best_test_acc:.4f}")
print(f"模型已保存到: {MODEL_SAVE_PATH}")
```

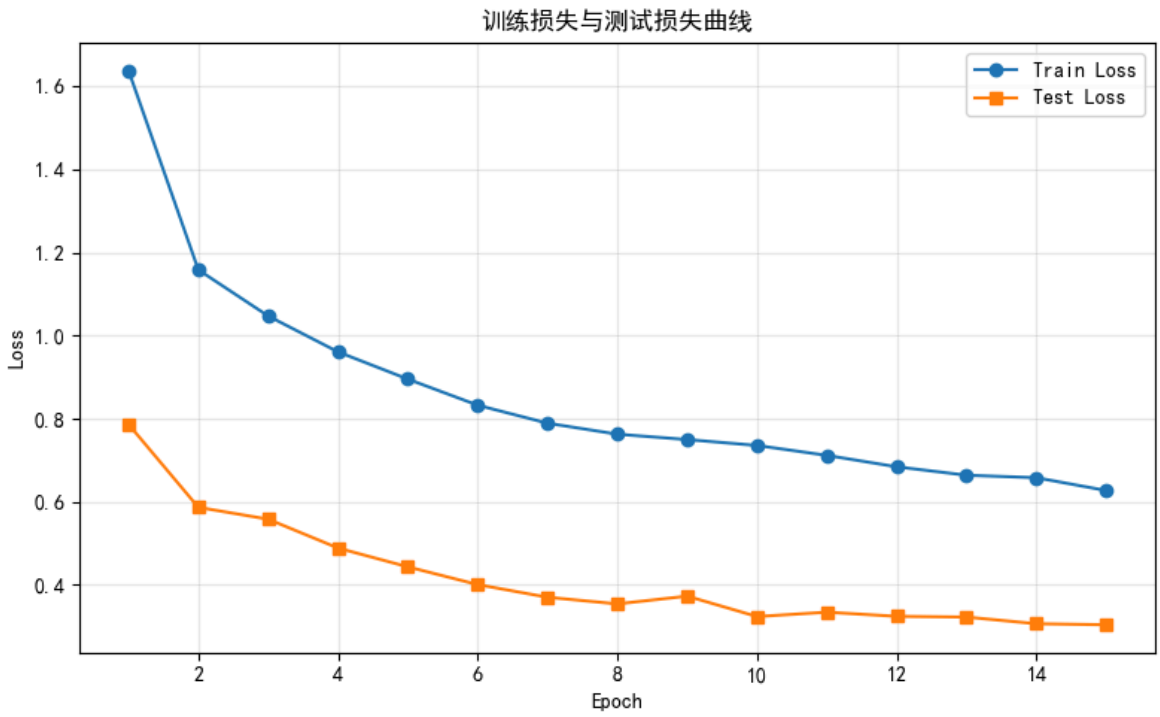
```
Epoch [1/15]
Train Loss: 1.6393 | Train Acc: 0.4137
Test  Loss: 0.7864 | Test  Acc: 0.7936
--------------------------------------------------
Epoch [2/15]
Train Loss: 1.1587 | Train Acc: 0.5804
Test  Loss: 0.5862 | Test  Acc: 0.8500
--------------------------------------------------
Epoch [3/15]
Train Loss: 1.0474 | Train Acc: 0.6268
Test  Loss: 0.5576 | Test  Acc: 0.8594
--------------------------------------------------
Epoch [4/15]
Train Loss: 0.9609 | Train Acc: 0.6598
Test  Loss: 0.4880 | Test  Acc: 0.8723
--------------------------------------------------
Epoch [5/15]
Train Loss: 0.8952 | Train Acc: 0.6812
Test  Loss: 0.4427 | Test  Acc: 0.8873
--------------------------------------------------
Epoch [6/15]
Train Loss: 0.8324 | Train Acc: 0.7063
Test  Loss: 0.4000 | Test  Acc: 0.8942
--------------------------------------------------
Epoch [7/15]
Train Loss: 0.7890 | Train Acc: 0.7241
Test  Loss: 0.3695 | Test  Acc: 0.9019
--------------------------------------------------
Epoch [8/15]
Train Loss: 0.7625 | Train Acc: 0.7337
Test  Loss: 0.3534 | Test  Acc: 0.9045
--------------------------------------------------
Epoch [9/15]
Train Loss: 0.7496 | Train Acc: 0.7400
Test  Loss: 0.3722 | Test  Acc: 0.9007
--------------------------------------------------
Epoch [10/15]
Train Loss: 0.7353 | Train Acc: 0.7440
Test  Loss: 0.3229 | Test  Acc: 0.9184
--------------------------------------------------
Epoch [11/15]
Train Loss: 0.7115 | Train Acc: 0.7514
Test  Loss: 0.3335 | Test  Acc: 0.9140
--------------------------------------------------
Epoch [12/15]
Train Loss: 0.6839 | Train Acc: 0.7665
Test  Loss: 0.3235 | Test  Acc: 0.9152
--------------------------------------------------
Epoch [13/15]
Train Loss: 0.6636 | Train Acc: 0.7752
Test  Loss: 0.3217 | Test  Acc: 0.9176
--------------------------------------------------
Epoch [14/15]
Train Loss: 0.6576 | Train Acc: 0.7763
Test  Loss: 0.3056 | Test  Acc: 0.9190
--------------------------------------------------
Epoch [15/15]
Train Loss: 0.6267 | Train Acc: 0.7906
Test  Loss: 0.3031 | Test  Acc: 0.9230
--------------------------------------------------
```

```
训练完成。
最佳测试准确率: 0.9230
模型已保存到: svhn_cnn_model.pth
```

In [12]:
```
# ========================
# 10. 绘制训练/测试准确率曲线
# ========================
epochs = range(1, NUM_EPOCHS + 1)

plt.figure(figsize=(8, 5))
plt.plot(epochs, train_accuracies, marker='o', label='Train Accuracy')
plt.plot(epochs, test_accuracies, marker='s', label='Test Accuracy')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("训练准确率与测试准确率曲线")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



In [13]:
```
# ========================
# 11. 绘制训练/测试损失曲线
# ========================
plt.figure(figsize=(8, 5))
plt.plot(epochs, train_losses, marker='o', label='Train Loss')
plt.plot(epochs, test_losses, marker='s', label='Test Loss')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("训练损失与测试损失曲线")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

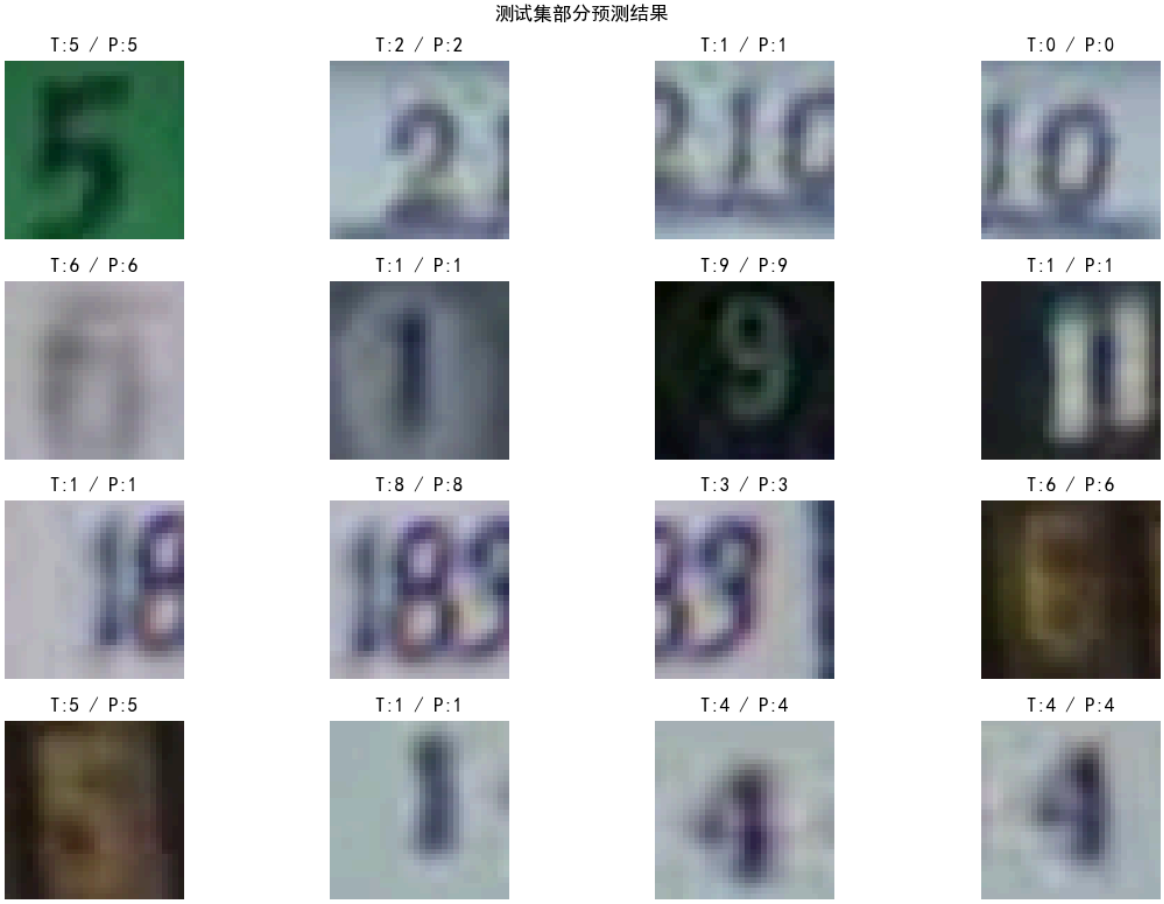


In [17]:
```
# ========================
# 12. 加载最佳模型并测试
# ========================
best_model = SimpleCNN(num_classes=NUM_CLASSES).to(device)
best_model.load_state_dict(torch.load(MODEL_SAVE_PATH, map_location=device, weig

final_test_loss, final_test_acc = evaluate(best_model, test_loader, criterion, d

print("=" * 60)
print("最终测试结果")
print("=" * 60)
print(f"Final Test Loss     : {final_test_loss:.4f}")
print(f"Final Test Accuracy : {final_test_acc:.4f}")
```

```
============================================================
最终测试结果
============================================================
Final Test Loss     : 0.3031
Final Test Accuracy : 0.9230
```

In [15]:
```
# ========================
# 13. 展示测试集部分预测结果
# ========================
best_model.eval()

images_batch, labels_batch = next(iter(test_loader))
images_batch = images_batch.to(device)
labels_batch = labels_batch.to(device)

with torch.no_grad():
    outputs = best_model(images_batch)
    _, preds = torch.max(outputs, 1)

images_np = images_batch.cpu().numpy()
labels_np = labels_batch.cpu().numpy()
preds_np = preds.cpu().numpy()

images_np = np.transpose(images_np, (0, 2, 3, 1))
```

```
images_np = (images_np * 0.5) + 0.5
images_np = np.clip(images_np, 0, 1)

plt.figure(figsize=(12, 8))
for i in range(16):
    plt.subplot(4, 4, i + 1)
    plt.imshow(images_np[i])
    plt.title(f"T:{labels_np[i]} / P:{preds_np[i]}")
    plt.axis("off")
plt.suptitle("测试集部分预测结果")
plt.tight_layout()
plt.show()
```



In [16]:

```
# ========================
# 14. 自动输出简要结论
# ========================
print("=" * 60)
print("实验总结")
print("=" * 60)
print("1. 已完成 SVHN Format 2 数据集的读取与预处理；")
print("2. 已使用 CNN 模型完成 0~9 共 10 类数字分类任务；")
print("3. 已输出测试集准确率（Test Accuracy）；")
print("4. 已绘制训练/测试准确率曲线；")
print("5. 已绘制训练/测试损失曲线；")
print("6. 已保存最佳模型权重文件。")
print(f"\n当前最佳 Test Accuracy 为：{best_test_acc:.4f}")
```

```
============================================================
实验总结
============================================================
1. 已完成 SVHN Format 2 数据集的读取与预处理；
2. 已使用 CNN 模型完成 0~9 共 10 类数字分类任务；
3. 已输出测试集准确率（Test Accuracy）；
4. 已绘制训练/测试准确率曲线；
5. 已绘制训练/测试损失曲线；
6. 已保存最佳模型权重文件。

当前最佳 Test Accuracy 为：0.9230
```